Localization of a Mobile Robot

Srikanth Nanda: 7026002
First examiner: Prof. Dr. Elmar Wings | Second examiner: Prof. Dr. Jonas Schwarz • Hochschule Emden/Leer

Problem Description

Localization allows a mobile robot to estimate its position and orientation within an environment. GPS is unreliable indoors, so robots must rely on alternative sensing such as RF, inertial, or **vision-based** methods.

This project investigates indoor localization on a Mobile robot using **monocular ORB-SLAM** with a Leopard Imaging IMX219 8MP camera and a 128-core NVIDIA Jetson Nano. The system must estimate pose and build a map using only camera data under indoor conditions.

Research Question:

Can monocular ORB-SLAM be reliably deployed end-to-end on an embedded NVIDIA Jetson Nano platform for real-time indoor robot localization?

Hardware Used:

- **Mobile Robot** Jetbot Kit and Sparkfun Qwiic Parts
- **NVIDIA Jetson Nano** (128-core GPU)
- **Leopard Imaging IMX219** monocular camera

Software Used:

- **ROS Melodic** on JetPack 4.3, **Python** 2 & 3, Keras, ORB-SLAM 2 & 3

Challenges

- **Scale Ambiguity**
  A single monocular camera cannot measure absolute depth; distances must be inferred via multi-view geometry.
- **Texture and Illumination**
  Feature-based methods rely on distinguishable texture. Low-texture surfaces, illumination changes, and occlusions reduce tracking reliability.
- **Dynamic Objects**
  Moving people or obstacles introduce outliers that can corrupt pose estimation if not rejected.
- **Embedded Compute Constraints**
  NVIDIA Jetson Nano has limited CPU/GPU resources, restricting the complexity of algorithms deployable in real time.
- **Integration Fragility**
  ORB-SLAM2/3, ROS Melodic (rosbuild), Python/Boost bindings, and JetPack 4.3 form a complex stack where small incompatibilities break end-to-end deployment.

Proposed Solution

Each challenge is addressed through a specific methodological decision:

- **Multi-View Geometry**
  Triangulation and bundle adjustment across multiple frames recover scale; loop closure corrects accumulated drift.
- **Robust Features (ORB)**
  ORB descriptors provide rotation, scale, and illumination invariance, improving matching in challenging indoor scenes.
- **Re-localization and Outlier Handling**
  Place recognition recovers from tracking failures; filtering reduces the impact of dynamic objects.
- **Embedded-Friendly SLAM**
  Keyframe-based SLAM and local map optimization balance accuracy and real-time performance on NVIDIA Jetson Nano.
- **Fallback Collision Avoidance**
  An ONNX-based collision-avoidance pipeline with ROS orchestration provides a practical navigation layer even when full monocular SLAM is not yet robust.

Approach	Pose estimate	Map / collision output	Compatibility with the Mobile Robot
ORB-SLAM2	Yes (visual SLAM)	Sparse feature map	NVIDIA Jetson Nano, ROS/bindings fragile
CNN-based	No (implicit)	Collision probability only	fully onboard and reactive only
CNN + ORB-SLAM2	Yes	Pose + CNN collision fusion	integration blocked on NVIDIA Jetson Nano
ORB-SLAM3	Yes (visual/VI SLAM)	Rich multi-map support	Best on ROS2/desktop, off-board for Mobile Robot

Table 1: Comparison of localization solutions proposed for the Mobile Robot platform.

Conclusion:

The thesis produced a well-structured onboard navigation stack and validated monocular ORB-SLAM on desktop datasets, but a fully stable end-to-end deployment on NVIDIA Jetson Nano under NVIDIA JetPack 4.3 and ROS Melodic was not yet achieved. The main limitation lies at the integration boundary between ORB-SLAM, ROS tooling, and Python/Boost bindings showing that hardware capability alone is not sufficient when ecosystem compatibility is fragile and violates NVIDIA policies.

System Architecture & Hardware



Figure 1: Mobile robot kit components

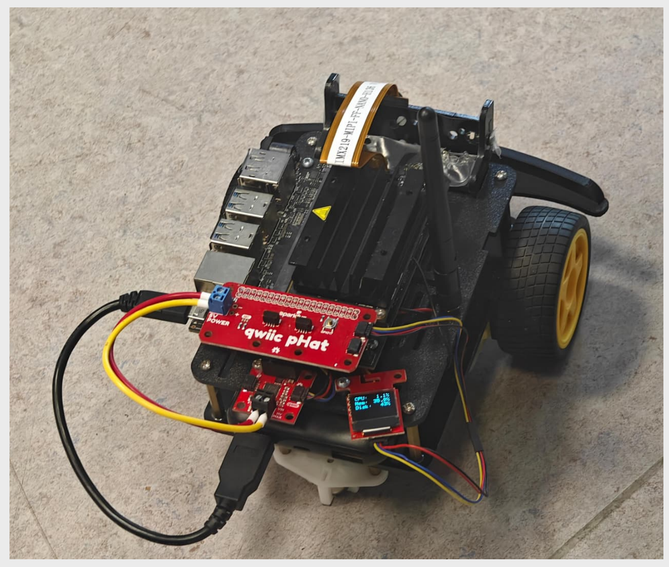


Figure 2: Assembled Mobile Robot platform featuring NVIDIA Jetson Nano and Leopard Imaging IMX219 H136 8MP Camera and Sparkfun Qwiic parts.

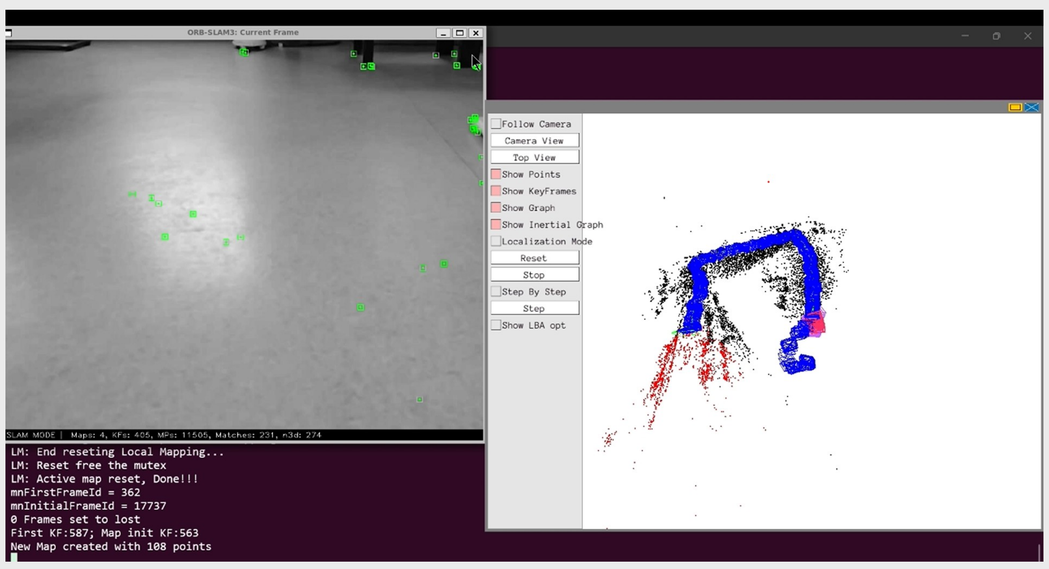


Figure 3: ORB-SLAM3 running and tracking in the current frame and reconstructed map points.

Results

Positive Outcomes:

- ORB-SLAM2 and ORB-SLAM3 built successfully as a standalone library
- Python navigation stack with occupancy grid and reactive decision node (slow/stop/recovery, heading stabilization)
- Functional ONNX collision-avoidance pipeline with ROS orchestration and telemetry logging

Integration Issues:

- Python bindings for ORB-SLAM2 failed undefined symbol in the Boost/OpenCV converter layer
- `rosbuild`-based examples could not obtain compile flags under JetPack 4.3 / ROS Melodic
- Launch referenced a missing `orb_slam2_ros` package; only `/rosout` remained active

Component	Status	Notes
ORB-SLAM2 core lib	**Built**	Standalone
Navigation stack	**Working**	ONNX-based
ROS integration	**Working**	rosbuild issue
Python bindings	Failed	Boost symbol

ORB-SLAM3 successfully tracked features and reconstructed a sparse 3D map on the TUM benchmark dataset, confirming the algorithm's viability. However, porting this pipeline onto the NVIDIA Jetson Nano's ROS Melodic environment exposed integration failures that prevented a stable onboard deployment.

LaTeX TikZposter